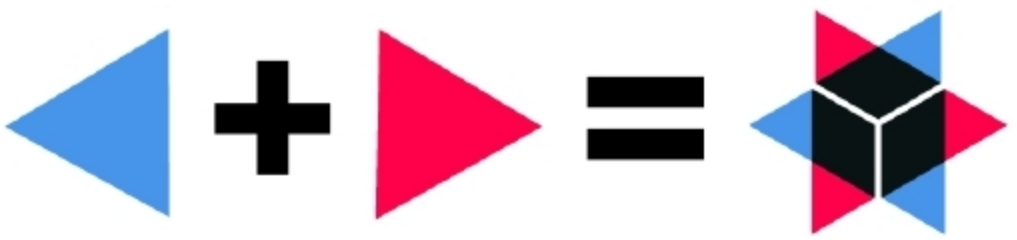
QUARKUS
Supersonic and Subatomic

Subatomic and supersonic! When I first saw those words in the May 2019 issue of *Open Source For You,* I thought that they had something to do with Avenger's Endgame spoilers. But on reading the article, I found that the words were applied to something even more interesting — Quarkus. Read on to learn more about it.

WHAT MAKES THE SUBATOMIC QUARK ELEMENT?



About two months back, Red Hat released Quarkus to get the developer community to rethink how Java can be best used to address a number of use cases. These include new deployment environments and application architectures like Kubernetes, microservices and serverless and cloud-native app development. The aim was to deliver higher levels of productivity and efficiency than the standard Java monolithic applications. According to Red Hat's release notes: "Quarkus is a Kubernetes native Java framework tailored for GraalVM and HotSpot, crafted from best-of-breed Java libraries and standards. The goal of Quarkus is to make Java a leading platform in Kubernetes and serverless environments while offering developers a unified reactive and imperative programming model to optimally address a wider range of distributed application architectures."

After reading a bit about Quarkus on its home page, I wanted to understand more about how it could totally change the way we package and deploy microservices. I decided to do a few experiments, to check how the three parameters change when a Java microservice is deployed using traditional openJDK vs Quarkus and GraalVM using containers. The results were unbelievable. We will go into that shortly, but before that, let me tell you more about GraalVM.

GraalVM

GraalVM is a high performance virtual machine that can run programs in different languages. Currently, it supports Java and other JVM languages like Scala, Kotlin, JavaScript, Python, Ruby and some native languages as well. For JVM languages, in particular, GraalVM offers a much faster runtime environment. You can learn more about it from the website *https://www.graalvm.org/*, but one of the limitations of GraalVM is that dynamic class loading is not supported. This means that deploying JARs, WARs, etc, in a container at runtime is impossible. Quarkus bridges that gap by providing a set of toolkits and frameworks for writing Java applications. It's light, cloud-friendly, designed for GraalVM, and helps to overcome the limitations of the latter, including the one mentioned above.

Now let's get to the experiments and results. On its blogs, Red Hat has often mentioned that Quarkus applications compile with GraalVM with tremendous memory utilisation and significantly low startup time, compared to traditional OpenJDK compilations and deployment. But I wanted to try it once to see how significant the difference was.

I started by bootstrapping a project and instead of the plain 'Hello, world' I had the GET service return a simple JSON using the Quarkus Extensions. Well, it does not actually matter how complicated the service is, but it feels